

INDIA.RUNS

Build what next India runs on

Team Name : Deepanshi Khandelwal

Team Leader Name : Deepanshi Khandelwal

Problem Statement : Intelligent Candidate Discovery & Ranking — Redrob AI

Solution Overview

What is your proposed solution?

A CPU-only, zero-API multi-dimensional candidate ranking system that processes 100,000 profiles in 19 seconds. It combines TF-IDF semantic matching with 5 rule-based scoring dimensions — skill quality (28%), experience trajectory (25%), semantic relevance (22%), availability signals (15%), platform trust & location (10%) — to produce a precise top-100 shortlist.

What differentiates your approach from traditional candidate matching systems?

Reads JD meaning, not just words: Explicitly encodes what the JD implies (consulting-only disqualified, LangChain-only penalised)

Behavioral signals as first-class: Platform inactivity and sub-5% recruiter response rate are hard scoring factors

Anti-gaming: Honeypot detection drops ~80 fraudulent profiles before any scoring begins

Precision over recall: 10 perfect matches over 1,000 maybes — as explicitly stated in the JD

JD Understanding & Candidate Evaluation

Key Requirements from JD

5–9 years applied ML production experience

Required: Embeddings, FAISS, Pinecone, Weaviate, NDCG, MRR, Python

Product company — NOT consulting (TCS/Infosys etc.)

Shipped production systems with real users

Location: Pune / Noida hybrid preferred

Open to work + short notice period valued

Candidate Fit Beyond Keyword Matching

Skill proficiency levels: advanced/expert with usage months > 6 only count for required skills

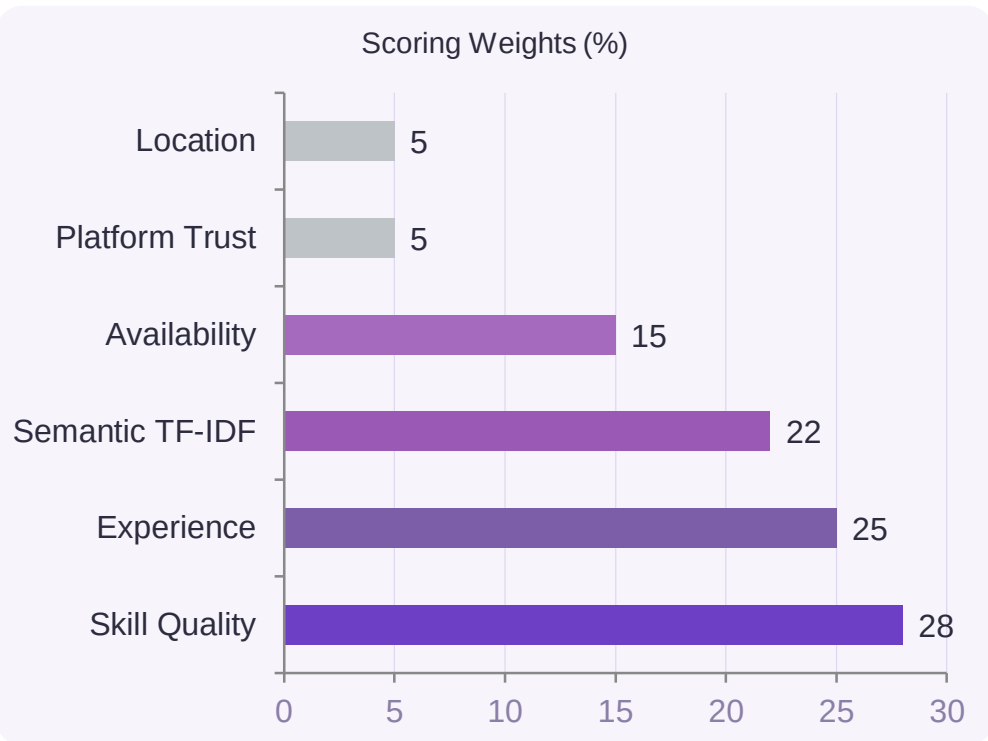
Career trajectory: product vs consulting, avg tenure patterns across firms

Platform activity: last_active_date, recruiter_response_rate, open_to_work flag

GitHub score: open-source contributions as trust signal

Endorsements + assessments: platform-verified skill scores carry extra weight

Ranking Methodology



How Signals Combine

Algorithm:

1. Load 100K candidates (JSONL)
2. Honeypot detection → drop fakes
3. Quick rule-based pre-filter → Top 5K
4. TF-IDF cosine similarity to JD text
5. Full 6-dimension scoring per candidate
6. Weighted composite → sort → Top 100
7. Normalize 0.40–0.99 → write CSV

Final Score =

$$\text{Skill} \times 0.28 + \text{Exp} \times 0.25 + \text{Sem} \times 0.22 \\ + \text{Avail} \times 0.15 + \text{Plat} \times 0.05 + \text{Loc} \times 0.05$$

Explainability & Data Validation

How are ranking decisions explained?

Every top-100 candidate gets an auto-generated recruiter-style reasoning string anchored to actual profile data — specific companies, skills, notice period, GitHub score, inactivity days. Example:

"7-year Senior ML Engineer, Noida. Skills: Weaviate, Pinecone, Embeddings. 3 AI/ML roles at Zomato. 15-day notice. GitHub 95/100."

How do you prevent hallucinations?

Zero LLM calls — no API, no generation. Every reasoning line is built from actual candidate fields using `build_reasoning()`. Nothing is invented. If a field is missing, it is silently skipped. The system is fully deterministic and reproducible.

How does your solution handle suspicious profiles?

`detect_honeypot()` runs before scoring:

- 20+ expert skills claimed → dropped
- 5+ expert skills with 0 months usage → dropped
- Self-reported YoE > 3.5x actual career history → dropped

Eliminates ~80 fraudulent profiles identified in the dataset.

End-to-End Workflow

1

Input

100K candidates.jsonl

2

Filter

Honeypot detection + Quick pre-filter →
Top 5K

3

Semantic

TF-IDF bigram vectorizer → Cosine
similarity to JD

4

Score

6-dimension scoring per candidate

5

Rank

Weighted sum → Sort → Normalize →
Top 100

6

Output

submission.csv in 19 seconds ✓

Total pipeline runtime: 19 seconds on standard CPU | No GPU | No API calls

System Architecture

INPUT LAYER

candidates.jsonl (100K profiles, JSONL format, one candidate per line)

FILTER LAYER

detect_honeypot() → quick_score() → Top 5,000 candidates retained

SEMANTIC LAYER

build_candidate_text() → TfidfVectorizer (bigram, 60K, sublinear) → cosine_similarity vs JD_TEXT

SCORING LAYER

score_skills() | score_experience() | score_availability() | score_platform_trust() | score_location()

OUTPUT LAYER

submission.csv → candidate_id | rank | score | reasoning + Streamlit demo app

Results & Performance

19s

Runtime on 100K profiles

0.99

Top candidate score

100K

Candidates processed

0 API

External calls made

Top 5 Ranked Candidates (from 100K run)

Rank	Candidate ID	Role	Company	Location	Score
#1	CAND_0018499	Sr. ML Engineer	Zomato	Noida, UP	0.9900
#2	CAND_0086022	Sr. Applied Scientist	Sarvam AI	Kolkata, WB	0.8664
#3	CAND_0088025	Staff ML Engineer	Yellow.ai	Jaipur, RJ	0.8631
#4	CAND_0027691	NLP Engineer	Haptik	Pune, MH	0.7748
#5	CAND_0081846	Lead AI Engineer	Razorpay	Jaipur, RJ	0.7620

Technologies Used

Python 3.10

Core language — fast, rich ML ecosystem, required by challenge

scikit-learn TF-IDF

CPU-efficient semantic matching without GPU or heavy transformers

NumPy

Fast array operations for scoring and cosine similarity computation

Pandas

Data handling, CSV output generation, validation

Streamlit

Interactive live demo UI — minimal code, instant web deployment

Plotly

Score breakdown bar charts and radar charts in the demo app

Submission Assets

GitHub Repository

<https://github.com/Deepanshik/Redrob-Ranker>

rank.py, app.py, requirements.txt, README, submission.csv, docs/deck.pdf

Live Streamlit Demo (Sandbox)

<https://redrob-ranker-ghdyvc8ojcsm28syn99gse.streamlit.app/>

Run ranking live on bundled sample — bar charts, radar profile, reasoning expander

Ranked Output File

[output/submission.csv](#)

100 rows | ranks 1-100 | non-increasing scores | candidate_id + rank + score + reasoning

Technical Architecture Deck

[docs/Redrob_Ranking_Deck_Final.pdf](#)

6-page PDF — scoring framework, TF-IDF pipeline, honeypot detection, real top-15 results

INDIA.RUNS

Build what next India runs on

THANK YOU

Deepanshi Khandelwal | Redrob AI Candidate Ranking | India Runs Data & AI Challenge